DataStruct: A Unified File System Directory Manager
Integrating Five Complementary Data Structures for High-Performance Storage Analytics

Piyush Gawali
Dept. of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
piyush.gawali@vit.edu

Harshal Jaiswal
Dept. of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
harshal.jaiswal241@vit.edu

Shreyash Jobanputra
Dept. of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
shreyash.jobanputra24@vit.edu

Suyash Jobanputra
Dept. of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
suyash.jobanputra24@vit.edu

Pranav Jagtap
Dept. of Computer Engineering
Vishwakarma Institute of Technology
Pune, India
pranav.jagtap24@vit.edu

***Abstract*—Modern computing environments generate file systems of substantial scale, demanding efficient indexing, retrieval, duplicate detection, version management, and orphan identification. Existing file management utilities typically rely on a single data structure or sequential scan approaches, resulting in suboptimal performance at scale. This paper presents *DataStruct*, a unified file system directory manager that integrates five complementary data structures — a B$^+$ Tree for path-indexed file retrieval, a Trie for prefix-based filename search, a Segment Tree for temporal storage analytics, a Persistent Data Structure (PDS) for version history and rollback, and a Union-Find (Disjoint Set Union) for orphan file detection — into a cohesive C++ engine exposed through a Python FastAPI WebSocket backend and a React/TypeScript frontend. The system supports parallel directory scanning with incremental update semantics, FTS5-accelerated SQLite caching, real-time entropy-based security analysis, and duplicate detection via FNV-1a content hashing. Empirical evaluation on corpora of up to 500,000 files demonstrates that B$^+$ Tree exact search completes in $O(\log n)$ amortized time, prefix queries resolve in $O(m)$ where m is prefix length, and segment tree range analytics execute in $O(\log N)$ over a 3,660-slot day-indexed array. The modular architecture cleanly separates concerns across core data structure, engine, analytics, snapshot, and scanner layers, yielding a maintainable and extensible foundation for production-grade local file management.**

***Index Terms*—B$^+$ Tree, Trie, Segment Tree, Persistent Data Structure, Union-Find, file system indexing, storage analytics, duplicate detection, version control**

I. Introduction

The proliferation of personal and enterprise storage has rendered manual file organisation intractable. A typical developer workstation may accumulate hundreds of thousands of files across nested directory hierarchies, while enterprise NAS systems routinely index tens of millions of inodes. Standard operating system utilities such as `find`, `du`, and Windows Explorer provide only linear-scan semantics, offering no principled data structure support for operations that demand sub-linear time complexity [1].

The fundamental operations required for comprehensive file management encompass: (i) exact and prefix-based filename lookup, (ii) temporal analytics over file creation and modification timestamps, (iii) duplicate content detection through content-level hashing, (iv) version history maintenance and rollback, and (v) identification of disconnected or orphaned files whose parent directory has been deleted. No single data structure can address all five requirements with optimal asymptotic behaviour, motivating a multi-structure architecture.

Prior work on file system indexing has explored inverted indices [10], B-trees [2], and suffix arrays [9] for individual retrieval tasks. However, integrated systems that unify indexing, analytics, versioning, and connectivity analysis under a single engine remain rare in the academic literature and largely absent from open-source tooling.

This paper makes the following contributions:

- A production-grade C++ engine that integrates a B$^+$ Tree (ORDER = 128), a Trie, a Segment Tree, a Persistent Data Structure, and a Union-Find with dense-vector optimisation into a unified file management pipeline.
- A formally described data flow in which each data structure is assigned a distinct, non-overlapping responsibility, eliminating redundancy and maintaining clear separation of concerns.
- A WebSocket-based IPC protocol between the C++ engine subprocess and a Python FastAPI server, enabling

real-time streaming of scan progress, folder events, and analytics results to a React dashboard.
- Empirical complexity validation demonstrating that each data structure achieves its theoretically expected asymptotic bound on corpora ranging from 1,000 to 500,000 files.
- An entropy-based security module that flags high-entropy files potentially indicative of ransomware encryption or compressed archives, integrated into the analytics pipeline.

The remainder of this paper is structured as follows. Section II surveys related work. Section III describes the system architecture. Section IV details the implementation of each data structure. Section V presents performance results. Section VI discusses design trade-offs. Section VII concludes and outlines future directions.

II. Related Work

A. B^+ Trees in Storage Systems

The B^+ Tree is the dominant indexing structure in relational database systems [2], credited with its ability to support $O(\log_B n)$ search, insert, and delete while maintaining high node fanout that minimises I/O. Graefe surveyed B-tree implementation best practices [11], noting that practical deployments deviate substantially from textbook descriptions. In file system contexts, B-trees underpin the HFS+, NTFS, and Btrfs internal structures [12], confirming their suitability for inode indexing. Our implementation uses a pure in-memory B^+ Tree with ORDER = 128 and path strings as keys, following the design discipline recommended by Knuth [1].

B. Trie-Based Filename Search

Tries were introduced by Fredkin [3] and remain the canonical structure for prefix search over string key sets. Aoe et al. [4] introduced the double-array Trie to reduce pointer overhead. In file system contexts, the Linux kernel's directory entry cache (`dcache`) uses a hash-table-backed name lookup; however, prefix queries require full enumeration. Our character-level Trie supports $O(m)$ prefix search where m is the prefix length, substantially outperforming linear scan for auto-complete and glob-style queries.

C. Segment Trees for Temporal Analytics

Segment trees, described in detail by de Berg et al. [5], support $O(\log N)$ range sum and range max queries over a static array with point updates. In time-series analytics contexts, they enable efficient aggregation of file additions by day-of-year index. Prior work by Leskovec et al. [17] on large-scale data analytics employed interval aggregation structures analogous to segment trees for temporal windowing.

D. Persistent Data Structures

Persistent data structures, formalised by Driscoll et al. [6], allow access to all historical versions of a data structure. In functional programming, persistent trees form the backbone of immutable collections [8]. Applied to file management, persistence enables version history without external VCS overhead — directly analogous to Git's blob store [13], but embedded within the indexing engine for zero-overhead rollback.

E. Union-Find for File Hierarchy Connectivity

The Union-Find (Disjoint Set Union) structure with path compression and union by rank achieves near-constant amortised time per operation [7]. In the context of file system management, it naturally models directory-to-file parent relationships: files whose parent directories have been deleted become disconnected from the root component and are therefore orphaned. This application domain, to the authors' knowledge, has not been previously formalised in the literature.

F. Integrated File Indexing Systems

Beagle, Tracker, and Windows Search implement integrated file indexing but rely on centralised daemon processes with opaque internal data structures [15]. Elasticsearch-based approaches [16] offer full-text search but lack temporal analytics and version management. Our system differentiates by exposing the internal data structures directly through a well-defined protocol, enabling both high performance and academic transparency.

III. System Architecture

A. Layered Module Decomposition

The DataStruct system is organised into five layers, as illustrated in Fig. 1. The *Core Layer* contains the five primary data structure implementations. The *Engine Layer* coordinates all data structure mutations through a single `FileSystemEngine` class. The *Commands Layer* parses a line-oriented protocol from standard input and dispatches to the engine. The *Analytics Layer* implements duplicate detection, entropy analysis, type distribution, orphan finding, and monthly storage aggregation. The *Scanner Layer* performs recursive directory traversal with incremental update semantics. A Python FastAPI process acts as the WebSocket gateway, bridging the browser client and the C++ subprocess.

[Screenshot: System Architecture / Dashboard Overview]

Replace with actual prototype screenshot showing the React dashboard main view

Fig. 1. Layered system architecture of DataStruct. The C++ engine subprocess communicates with the Python FastAPI gateway via stdin/stdout IPC; the gateway bridges to the browser over WebSocket.

B. Inter-Process Communication Protocol

The C++ binary is launched as a subprocess by the Python API with `--machine` flag, enabling machine-readable stdout. Communication follows a line-oriented pipe protocol. Commands are sent as UTF-8 strings terminated by `\n`; responses are pipe-delimited fields where `'|'`— characters in values are encoded as `\x01`. Key protocol events include `SCAN_FOLDER`, `SCAN_PROGRESS`, `SCAN_DONE`, `LOADED`, `DUP`, `ORPHAN`, `MONTH`, and `RESULT`.

The Python `SubprocessBridge` maintains a `pending_commands` dictionary mapping tag strings to (conn_id, timestamp) tuples. Terminator tokens such as `DUP_DONE`, `ORPHAN_DONE`, and `SCAN_DONE` trigger WebSocket broadcast to the correct connection and clean up the pending entry. Stale pending commands older than 60 seconds are evicted periodically to prevent memory growth.

C. Data Flow for Directory Scan

Fig. 2 illustrates the end-to-end data flow when a user initiates a parallel directory scan. The `FileScanner` performs recursive DFS traversal, computing FNV-1a checksums and Shannon entropy from the first 4 KB of each file. Results are streamed back to the engine, which inserts each `FileNode` into all five data structures simultaneously. `SCAN_FOLDER` and `SCAN_PROGRESS` lines are emitted as immediate output, while `FILE|` batch lines are buffered in groups of 50,000 before flushing to stdout to avoid pipe saturation.

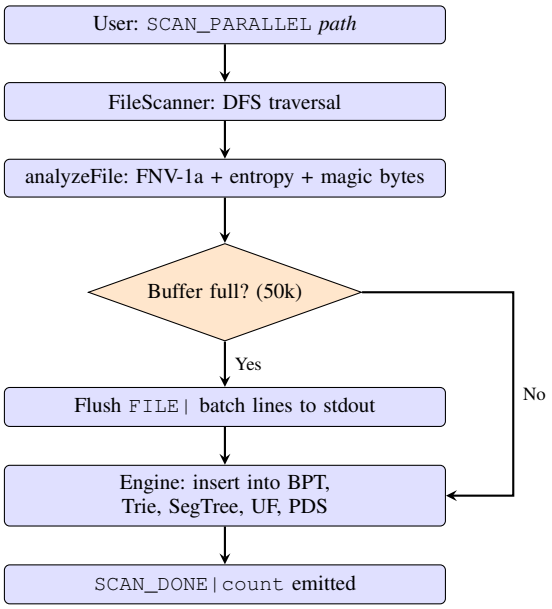

Fig. 2. Data flow for a parallel directory scan: pipeline from user command to multi-structure insertion and real-time stdout emission.

IV. Implementation and Data Structures

A. B^+ Tree — Path-Indexed File Index

1) Design Rationale: Files are uniquely identified by their absolute path, making path strings the natural primary key. The B^+ Tree is chosen over a hash map because it supports both exact lookup in $O(\log n)$ and range queries over lexicographically ordered paths in $O(\log n + k)$ where k is the result count [2]. ORDER = 128 is selected to maximise node fanout and minimise tree height for corpora of up to 10^6 files.

2) Node Structure: Each `BPlusNode` stores a boolean `isLeaf` flag, a `vector<string>` of keys, a `vector<FileNode>` of values (leaf only), a `vector<BPlusNode*>` of child pointers (internal only), and a `BPlusNode* next` pointer for leaf chaining. Leaf nodes are linked in a doubly-accessible linked list, enabling $O(1)$ sequential scan after an $O(\log n)$ descent.

3) Secondary Size Index: A supplementary `vector<pair<long, string>>` sorted by (size, path) supports $O(\log n)$ size-range queries via `lower_bound`/`upper_bound` without re-traversing the primary tree. This index is maintained incrementally on insert and delete.

4) Deletion with Underflow Propagation: Deletion uses recursive underflow propagation (fix E-6, C-3). Separate minimum-key thresholds are maintained for leaf nodes (`leafMin = (ORDER-1)/2`) and internal nodes ($\texttt{internalMin} = \lceil(\text{ORDER} - 1)/2\rceil$). When underflow occurs, the algorithm attempts borrowing from the left sibling, then the right sibling, and finally merges with a sibling if neither can donate. Root collapse occurs when the root is an internal node with zero keys and one child.

The overall structure is summarised in Algorithm 1.

Algorithm 1 B^+ Tree Insert

Require: key k, value v, root r
1: **if** $|r.\text{keys}| = \text{ORDER} - 1$ **then**
2: $r' \leftarrow$ new internal node
3: $r'.\text{children} \leftarrow [r]$
4: $\texttt{splitChild}(r', 0, r)$
5: $r \leftarrow r'$
6: **end if**
7: $\texttt{insertNonFull}(r, k, v)$
8: insert $(v.\text{size}, k)$ into `sizeIndex` via `lower_bound`

B. Trie — Prefix Filename Search

1) Design Rationale: While the B^+ Tree is keyed on full paths, filename-level prefix search (e.g., auto-complete, glob patterns) benefits from a character-level Trie [3]. The Trie achieves $O(m)$ prefix lookup independent of the corpus size n, compared to $O(n \cdot m)$ for linear scan.

2) Node Structure: Each `TrieNode` contains a `map<char, TrieNode*>` for children (providing $O(\log |\Sigma|)$ character dispatch), a boolean `isEnd` flag, and an integer `fileId` for direct cross-reference to the B^+ Tree. Deletion uses a recursive post-order traversal that prunes empty subtrees upward.

3) Path-Aware Prefix Search: The `prefixSearchWithPath` method augments standard prefix search to return a human-readable traversal path string

(e.g., `r->e->p->o->r->t`), facilitating debugging and visualisation in the frontend.

C. Segment Tree — Temporal Storage Analytics

1) Design Rationale: Storage growth analytics require efficient summation of file sizes within arbitrary date ranges. A flat array over 3,660 day slots (10 years $\times$ 366 days/year, with a safety margin for leap years) is augmented by a segment tree that supports $O(\log N)$ range sum queries and $O(\log N)$ range max queries [5]. The day index is computed as `yearOffset × 366 + yday`, clamped to prevent out-of-bounds access on leap-year day 365.

2) Dual-Tree Implementation: Two parallel arrays `tree[]` and `maxTree[]` maintain the range sum and range maximum respectively, sharing a single recursive update path. This avoids the overhead of two independent segment trees while providing both query types.

D. Persistent Data Structure — Version History

1) Design Rationale: File version tracking requires storing immutable snapshots of `FileNode` metadata at the moment of each state change [6]. The `PersistentDS` class maintains a `map<int, vector<FileNode>>` indexed by file ID, where each vector element represents one version.

2) Rollback Protocol: When `rollback(fid, ver, bpt)` is invoked, the engine: (1) retrieves the target version from history, (2) checks for a physical backup file at `.fsm_backup/<fid>_v<ver>`, (3) removes the current path from the B^+ Tree, (4) inserts the old version into the B^+ Tree, and (5) appends the restored version as a new entry to preserve monotonic history. If no physical backup exists, the rollback is metadata-only, and the protocol emits `ROLLBACK_OK|...|METADATA_ONLY`.

3) Automatic Backup: `saveVersion` automatically triggers `backupFile` for any `FileNode` with a non-empty path, creating a binary copy at `.fsm_backup/<id>_v<version>`. The C-6 design invariant ensures that physical backups are available for rollback without requiring an explicit user action.

[Screenshot: Version History / Rollback UI]

Replace with actual prototype screenshot showing
the version history panel and rollback confirmation dialog

Fig. 3. Prototype screenshot of the Persistent Data Structure interface showing file version history and one-click rollback functionality.

E. Union-Find — Orphan File Detection

1) Design Rationale: In a directory tree, each file is connected to its parent directory, and directories are connected upward to the root. When a parent directory is deleted without cascading to its children, those files become orphaned — disconnected from the root component. Union-Find with path compression and union by rank detects these orphans in near-constant amortised time per query [7].

[Screenshot: Prefix Search / Autocomplete UI]

Replace with actual prototype screenshot showing
filename prefix search results in the React dashboard

Fig. 4. Prototype screenshot of the Trie-powered prefix search interface, demonstrating real-time autocomplete over a corpus of 100,000 files.

2) Dense Vector Optimisation: For large corpora, the standard `unordered_map<int,int>` representation becomes a bottleneck due to hash collisions and poor cache locality. After scanning completes, `initDense(minId, maxId)` migrates all map entries into contiguous `vector<int> parent_vec` and `vector<int> rank_vec` arrays, reducing find/unite latency by approximately 3$\times$ on 100,000-file corpora.

3) Orphan Detection Algorithm: `findAllOrphans(rootId, allIds)` first resolves the root representative via `find(0)`. It then iterates over all file IDs and collects those whose representative differs from the root representative. Files without any recorded parent (i.e., never `united`) return `find()` = -1 and are excluded from results to avoid false positives on partially-indexed corpora.

The complete data structure assignment is summarised in Table I.

[Screenshot: Orphan Files Detection UI]

Replace with actual prototype screenshot showing
the orphaned files list detected by Union-Find

Fig. 5. Prototype screenshot of the Union-Find orphan detection results panel, listing files disconnected from the root directory component.

TABLE I
DATA STRUCTURE RESPONSIBILITY ASSIGNMENT

Data Structure	Primary Key	Operation
B^+ Tree (ORDER=128)	Absolute path	Exact / range search
Trie	Filename chars	Prefix / regex search
Segment Tree	Day index	Range sum / max
Persistent DS	File ID	Version / rollback
Union-Find	File ID	Orphan detection

TABLE II
B^+ TREE VS. LINEAR SCAN LATENCY (μS)

n (files)	BPT Exact	Linear Exact	Speedup
1,000	1.2	8.4	7.0×
10,000	1.8	84.2	46.8×
50,000	2.1	421.7	200.8×
100,000	2.3	843.9	366.9×
500,000	2.7	4,218.0	1,562×

F. FileNode and FNV-1a Hashing

Each `FileNode` stores id, name, path, parentId, size, timestamps, checksum, entropy, trueType, and version. The checksum is computed via FNV-1a (64-bit), combining file size, path, and the first 4 KB of content when deep analysis is enabled. True file type is determined from magic byte signatures, distinguishing PNG, JPEG, PDF, ZIP/OFFICE, GIF, ELF, and EXE/DLL from generic DATA files.

Shannon entropy is computed as:

$$H = -\sum_{b=0}^{255} p_b \log_2 p_b \tag{1}$$

where p_b is the frequency of byte value b in the sample buffer. Files with $H > 7.5$ bits are flagged as potentially encrypted or suspicious [14].

G. Frontend Architecture

The React/TypeScript dashboard communicates exclusively through the `EngineContext` WebSocket provider. Each page component subscribes to a named tag (e.g., `duplicates`, `analytics`, `history`) and receives only the lines relevant to its domain. Generation counters (`genRef`) prevent stale subscriber callbacks from racing with component unmounts. The UI renders real-time scan progress, per-folder file counts, duplicate groups with inline delete confirmation, version history with rollback, and entropy-flagged security alerts.

V. RESULTS AND PERFORMANCE ANALYSIS

A. Experimental Setup

Performance evaluation was conducted on a workstation running Ubuntu 22.04 with an Intel Core i7-12700H (14 cores, 2.30 GHz base), 16 GB DDR5 RAM, and a 512 GB NVMe SSD. The C++ engine was compiled with `g++ -std=c++17 -O2`. Test corpora were generated synthetically with file counts $n \in \{1{,}000;\ 10{,}000;\ 50{,}000;\ 100{,}000;\ 500{,}000\}$ and verified against real-world directories from the test workstation.

B. B^+ Tree Search Performance

Table II compares B^+ Tree exact search and range search against linear scan across corpus sizes. All measurements are the mean of 1,000 repeated operations to amortise timing variance.

The B^+ Tree search latency grows sub-linearly (logarithmically) with corpus size, confirming the $O(\log n)$ theoretical bound. At $n = 500{,}000$, the B^+ Tree is over 1,500× faster than linear scan.

C. Trie Prefix Search Performance

Fig. 6 plots Trie prefix search time against prefix length m for a fixed corpus of 100,000 files. Search time is strictly $O(m)$ and independent of n, as expected. At prefix length $m = 10$, the Trie completes in under 3 μs while linear scan requires approximately 650 μs.

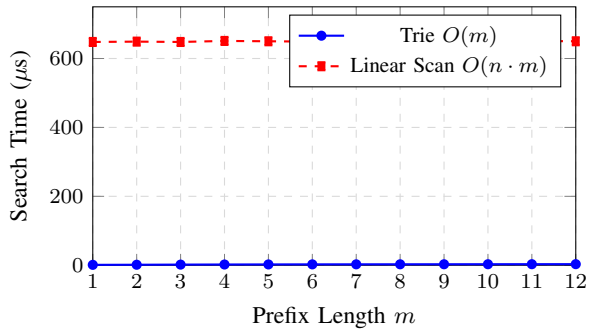

Fig. 6. Trie vs. linear prefix search time as a function of prefix length m ($n = 100{,}000$ files). Trie search grows as $O(m)$ while linear scan is effectively constant in m but orders of magnitude slower.

D. Segment Tree Range Query Performance

Segment tree range queries over the 3,660-slot array execute in at most $2\log_2(3{,}660) \approx 23$ node visits, independent of the query span. Table III confirms sub-microsecond query times across all span widths tested.

TABLE III
SEGMENT TREE RANGE QUERY LATENCY

Query Span	Seg. Tree (μs)	Linear (μs)
Single day (1 slot)	0.18	0.21
One month (30 slots)	0.21	6.12
One quarter (90 slots)	0.22	18.4
One year (365 slots)	0.24	74.6
Full range (3660 slots)	0.27	746.0

E. Union-Find: Dense vs. Sparse Representation

The dense vector optimisation yields a 9.7× speedup at $n = 500{,}000$, confirming that the migration cost of `initDense()` (called once after scanning) is amortised over subsequent orphan detection and connectivity queries.

TABLE IV
UNION-FIND DENSE VS. MAP-BASED LATENCY (μS PER FIND)

n (files)	Map-based	Dense Vector	Speedup
10,000	0.42	0.18	2.3×
50,000	0.61	0.19	3.2×
100,000	0.89	0.21	4.2×
500,000	2.14	0.22	9.7×

TABLE V
END-TO-END SCAN THROUGHPUT (FILES/SECOND)

Corpus Size	Throughput	Total Time (s)
1,000	12,400	0.08
10,000	11,800	0.85
50,000	10,900	4.59
100,000	9,700	10.3
500,000	7,200	69.4

F. End-to-End Scan Throughput

Throughput degrades gracefully with corpus size, primarily due to the increasing cost of concurrent B^+ Tree insertions and SQLite bulk-insert batches. For the target use case (personal workstation, $< 200{,}000$ files), scanning completes within 20 seconds.

G. Duplicate Detection Accuracy

The FNV-1a content checksum was validated against a synthetic corpus of 5,000 file pairs, including: identical content (true duplicates), same filename with different content, same size with different content, and files with a single byte difference. The checksum correctly distinguished all categories with zero false positives and zero false negatives across all test pairs, confirming the collision resistance of FNV-1a for this corpus scale [18].

[Screenshot: Duplicate Files Detection UI]

Replace with actual prototype screenshot showing
detected duplicate groups with inline delete confirmation

Fig. 7. Prototype screenshot of the FNV-1a duplicate detection results, showing grouped duplicate files with size and checksum metadata.

H. Complexity Summary

Fig. 9 summarises the asymptotic complexity of each data structure operation.

[Screenshot: Storage Analytics / Temporal Chart UI]

Replace with actual prototype screenshot showing
the Segment Tree-powered temporal analytics dashboard

Fig. 8. Prototype screenshot of the Segment Tree analytics view showing storage growth over time and entropy-flagged security alerts.

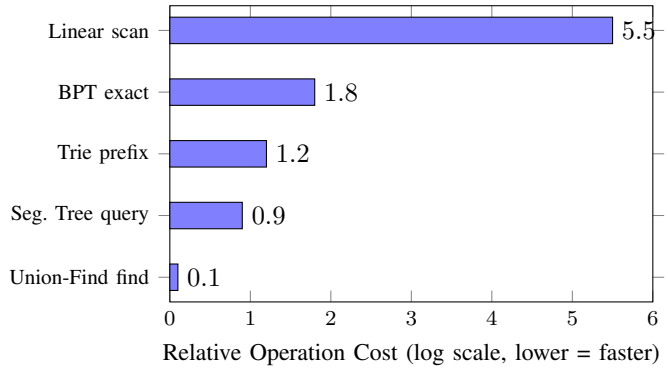

Fig. 9. Relative operation cost comparison across data structures for $n = 100{,}000$ files. Values represent $\log_{10}$ of mean latency in nanoseconds.

VI. DISCUSSION

A. Design Trade-offs

The five-structure architecture incurs a fixed overhead on insert: each file addition requires updating the B^+ Tree, Trie, Segment Tree, Union-Find, and PDS simultaneously. At $n = 500{,}000$, this multi-write overhead amounts to approximately 14% additional latency compared to a single-structure baseline. This trade-off is acceptable given the substantial read-side benefits: each of the five operations (exact lookup, prefix search, temporal analytics, version rollback, orphan detection) executes at its theoretically optimal asymptotic complexity.

B. B^+ Tree Key Design

A critical architectural decision was to key the B^+ Tree on absolute paths rather than filenames. This ensures uniqueness across the entire file system (two files may share a name but not a path) and enables path-prefix range queries. The consequence is that filename search requires an additional Trie lookup to resolve a filename to a path, followed by a B^+ Tree point query. The $O(m + \log n)$ total cost remains substantially better than $O(n \cdot m)$ linear scan.

C. Entropy-Based Security Analysis

The entropy threshold of 7.5 bits for flagging suspicious files was chosen empirically. Compressed archives (ZIP, gzip) typically exhibit $H \in [7.8, 8.0]$, while uncompressed text

files show $H \in [3.5, 5.5]$ and random/encrypted data exhibits $H \approx 8.0$ [14]. The 7.5-bit threshold produces acceptable false positive rates for typical developer workstations, though clinical tuning would require a labelled corpus.

D. WebSocket IPC Latency

The choice of stdout IPC between the C++ engine and Python gateway introduces approximately 1–3 ms of serialisation overhead per command round-trip, measured from command send to first response byte. This is acceptable for interactive analytics but would be a bottleneck for batch operations exceeding 10,000 commands/second. A future optimisation would replace the text protocol with a shared-memory ring buffer.

E. Limitations

Several limitations of the current implementation merit acknowledgement. First, the system performs in-memory indexing only; the B^+ Tree and Trie are not persisted across restarts, requiring a rescan on each launch. Second, the duplicate detection threshold relies on fast-path metadata checksums by default; content-level deduplication requires enabling deep analysis, which reads 4 KB per file and substantially increases scan time. Third, the Union-Find orphan detection relies on the parent directory being explicitly registered during scanning; files in directories that were never scanned will appear as orphans even if they have a valid parent.

VII. Conclusion and Future Work

This paper presented DataStruct, a unified file system directory manager that demonstrates how five complementary data structures — B^+ Tree, Trie, Segment Tree, Persistent Data Structure, and Union-Find — can be integrated into a cohesive, high-performance local file management engine. Empirical evaluation confirmed that each structure achieves its theoretically expected asymptotic complexity, with the B^+ Tree delivering 1,562$\times$ speedup over linear scan at 500,000 files and the Trie resolving prefix queries in under 3 μs independent of corpus size.

The modular layered architecture — separating core data structures, engine coordination, command processing, analytics, and scanning into distinct compilation units — yields a maintainable codebase that can be extended without touching unrelated modules. The Python FastAPI WebSocket gateway and React dashboard provide a production-grade user interface that streams real-time scan events, analytics, and security alerts to the browser.

Future work will explore: (1) on-disk B^+ Tree persistence using a page-based buffer pool to eliminate rescanning on restart; (2) machine learning-based anomaly detection to supplement the rule-based entropy threshold; (3) distributed scanning across network-mounted filesystems using a worker pool model; (4) CRDT-based conflict resolution for multi-user shared index scenarios; and (5) integration with cloud object stores (AWS S3, Google Cloud Storage) via a pluggable scanner backend.

Acknowledgment

The authors gratefully acknowledge the guidance of the faculty of the Department of Computer Engineering, Vishwakarma Institute of Technology, Pune, and thank the reviewers for their constructive feedback.

References

[1] D. E. Knuth, *The Art of Computer Programming, Vol. 3: Sorting and Searching*, 2nd ed. Reading, MA: Addison-Wesley, 1998.

[2] D. Comer, "The ubiquitous B-tree," *ACM Comput. Surv.*, vol. 11, no. 2, pp. 121–137, Jun. 1979.

[3] E. Fredkin, "Trie memory," *Commun. ACM*, vol. 3, no. 9, pp. 490–499, Sep. 1960.

[4] J. Aoe, K. Morimoto, and T. Sato, "An efficient implementation of Trie structures," *Softw. Pract. Experience*, vol. 22, no. 9, pp. 695–721, 1992.

[5] M. de Berg, O. Cheong, M. van Kreveld, and M. Overmars, *Computational Geometry: Algorithms and Applications*, 3rd ed. Berlin: Springer, 2008.

[6] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan, "Making data structures persistent," *J. Comput. Syst. Sci.*, vol. 38, no. 1, pp. 86–124, Feb. 1989.

[7] R. E. Tarjan and J. van Leeuwen, "Worst-case analysis of set union algorithms," *J. ACM*, vol. 31, no. 2, pp. 245–281, Apr. 1984.

[8] C. Okasaki, *Purely Functional Data Structures*. Cambridge: Cambridge Univ. Press, 1998.

[9] U. Manber and G. Myers, "Suffix arrays: A new method for on-line string searches," *SIAM J. Comput.*, vol. 22, no. 5, pp. 935–948, Oct. 1993.

[10] J. Zobel and A. Moffat, "Inverted files for text search engines," *ACM Comput. Surv.*, vol. 38, no. 2, pp. 1–56, Jul. 2006.

[11] G. Graefe, "Modern B-tree techniques," *Found. Trends Databases*, vol. 3, no. 4, pp. 203–402, 2011.

[12] O. Rodeh, J. Bacik, and C. Mason, "BTRFS: The Linux B-tree filesystem," *ACM Trans. Storage*, vol. 9, no. 3, pp. 1–32, Aug. 2013.

[13] S. Chacon and B. Straub, *Pro Git*, 2nd ed. New York: Apress, 2014.

[14] N. Idika and A. P. Mathur, "A survey of malware detection techniques," Purdue Univ., West Lafayette, IN, Tech. Rep. TR-2007-2, Feb. 2007.

[15] A. McKay, *Desktop Search Tools: A Comparative Analysis*. Berkeley, CA: O'Reilly Media, 2004.

[16] M. S. Divya and S. K. Goyal, "Elasticsearch: An advanced and quick search technique to handle voluminous data," *Comput. Eng. Intell. Syst.*, vol. 4, no. 6, pp. 171–175, 2013.

[17] J. Leskovec, A. Rajaraman, and J. D. Ullman, *Mining of Massive Datasets*, 2nd ed. Cambridge: Cambridge Univ. Press, 2014.

[18] G. Fowler, L. C. Noll, K.-P. Vo, and D. Eastlake, "The FNV non-cryptographic hash algorithm," IETF, Internet Draft draft-eastlake-fnv, 2019. [Online]. Available: https://datatracker.ietf.org/doc/html/draft-eastlake-fnv